UNIDAD 7 · DEL CRUD AL E-COMMERCE · CLASE 1

Tu tienda online

El TP integrador arranca hoy: consigna, grupos, rubros y la base de datos de tu tienda

Express | Supabase | JavaScript | Vercel

Diseño Backend · 5.º año · Prof. Julia Panei

LO QUE YA SABEN

De la pizzería a la tienda

En la pizzería aprendieron...		...y en la tienda lo usan para
Capas: routes, controllers, models	→	La misma estructura, con más archivos
CRUD contra Supabase	→	Catálogo, pedidos y stock
El total lo calcula el backend	→	El precio sale de la base, nunca del carrito
Validar y responder 400 / 404	→	Sin stock, carrito vacío, producto inexistente

Lo nuevo: usuarios con login, fotos de productos y el sitio publicado en internet.

LA CONSIGNA

Lo que tu tienda tiene que hacer

✓ Catálogo

productos con foto, precio y stock, con filtro por categoría

✓ Carrito

agregar, quitar y cambiar cantidades

✓ Usuarios

registro y login con Supabase Auth

✓ Compra

el backend controla el stock y calcula el total

✓ Online

publicada en Vercel, con un link que funcione

✓ README

integrantes, link del deploy y cómo se usa

PARA MEJOR NOTA

Los extras

+ **Panel admin**

ABM de productos con rol admin; un cliente recibe 403

+ **Buscador**

buscar productos por nombre con .ilike()

+ **Mis pedidos**

cada usuario ve sus compras con sus items

+ **Detalle de producto**

una página por producto, con 404 si no existe

Regla de oro: primero el mínimo funcionando y online. Recién después, los extras.

GRUPOS Y RUBROS

Cada grupo, una tienda distinta

Cómo se arman

- Grupos de 2 o 3 integrantes.
- El rubro se sortea y no se repite en la comisión.
- Todos programan: en la defensa cualquiera puede explicar cualquier parte.

Librería	**Vivero**	**Juguetería**
Ferretería	**Café en grano**	**Mascotas**
Camping	**Instrumentos**	**Cosmética natural**
Bicicletería	**Ropa deportiva**	**Panadería**

NO ARRANCAN DE CERO

La plantilla de la tienda

Ya viene hecho

- La estructura en capas
- La página, los estilos y el carrito
- Los controllers y las rutas
- El script de las tablas
- El README con el checklist

Lo completan ustedes

Clase 2: los models y el middleware

Clase 3: el fetch del front

Mientras tanto, cada parte que falta responde `501 Falta implementar`. Es lo esperado.

LA SEGUNDA MITAD DE HOY

La base de tu tienda, en cinco pasos

1. **Proyecto** — en Supabase
2. **Tablas** — con el script
3. **Tu rubro** — categorías y productos
4. **Imágenes** — en Storage
5. **Plantilla** — corriendo

El paso 3 es el que hace que la tienda sea suya: sin productos de su rubro, no hay tienda.

PASO 1

Un proyecto nuevo para la tienda

1. **New project**
   Nombre: tienda-[rubro]
2. **Región**
   South America (São Paulo)
3. **Contraseña de la base**
   Generala y guardala: no se vuelve a mostrar
4. **Project Settings → API Keys**
   Copiá la URL y la clave secreta

```
# .env (nunca se sube a GitHub)
SUPABASE_URL=https://xxxx.supabase.co
SUPABASE_KEY=tu_clave_secreta
```

¿Por qué la secreta?

Porque solo la usa tu servidor. Con ella, Express entra aunque las tablas estén cerradas al público. Por eso jamás va al front, ni a GitHub, ni a una captura.

PASO 2

Las tablas de la tienda

categorias

```
id
nombre
```

auth.users

```
id (uuid)
email
(la maneja Auth)
```

perfiles

```
id → auth.users
nombre
rol
```

productos

```
id
nombre, descripcion
precio, stock
imagen_url
categoria_id → categorias
activo
```

pedidos

```
id
usuario_id → auth.users
total
estado ('pendiente')
creado (fecha)
```

pedido_items

```
id
pedido_id → pedidos
producto_id → productos
cantidad
precio_unitario
```

→ clave foránea

Están en db/esquema.sql de la plantilla: se pegan en el SQL Editor y Run.

PASO 2 · LA PREGUNTA CLAVE

¿Para qué guardar el precio en cada item?

Hoy

Ana compra 2 libros a $20.000. Pedido total: $40.000.

El mes que viene

El libro sube a $25.000.

Si no guardaste precio_unitario...

el pedido viejo de Ana calcularía 2 × $25.000 = $50.000. Pero ella pagó $40.000. La base estaría contando una historia falsa.

precio_unitario es una foto del precio en el momento de la compra.

PASO 3

Los datos de tu rubro

Ejemplo: una librería

```
insert into categorias (nombre) values
  ('Novelas'), ('Infantiles'), ('Útiles escolares');

insert into productos
  (nombre, descripcion, precio, stock, categoria_id) values
  ('El Eternauta', 'Edición integral', 32000, 6, 1),
  ('Cuentos de la selva', 'Ilustrado', 14500, 10, 2),
  ('Cuaderno A4 rayado', '84 hojas', 3900, 40, 3),
  ('Caja de lápices x12', 'Colores', 5200, 1, 3);
```

Las reglas

- Al menos 3 categorías
- Al menos 8 productos
- Precios y descripciones creíbles
- Uno con stock bajo (1 o 0), para probar el "sin stock"

categoria_id es el id de la categoría: el orden del insert de categorías importa.

PASO 4

Las fotos van a Storage

1. **Storage → New bucket**

   Nombre: productos. Marcalo como Public.

2. **Nombrá las fotos con el id**

   1.jpg, 2.jpg, 3.jpg... según el id de cada producto

3. **Subilas al bucket**

   Arrastrá los archivos. Livianas: menos de 500 KB

4. **Un UPDATE para todas**

   Con la URL base de tu proyecto

```
update productos
set imagen_url =
  'https://xxxx.supabase.co/storage/v1'
  || '/object/public/productos/'
  || id || '.jpg';
```

`||` une textos en SQL. Para el producto 3 arma `.../productos/3.jpg`. Fotos propias o de bancos libres, como Unsplash o Pexels.

PASO 5

La plantilla corriendo

```
# 1. Descomprimir el .zip y entrar a la carpeta
cd tienda-plantilla

# 2. Crear el .env copiando el modelo y completarlo
cp .env.example .env

# 3. Instalar y levantar
npm install
npm start
```

Abrí localhost:3000

Vas a ver la tienda con tus categorías y un aviso:

`Falta implementar: cargarProductos`

Es lo esperado. Los productos aparecen en la Clase 3.

En Windows, en vez de cp: copy .env.example .env

ANTES DE IRSE

El checklist de hoy

- [ ] **Proyecto creado**
  y todos los integrantes con acceso
- [ ] **Cinco tablas**
  visibles en el Table Editor
- [ ] **Datos del rubro**
  3 categorías y 8 productos o más
- [ ] **Fotos**
  cada producto con su imagen_url abriendo la foto
- [ ] **Plantilla**
  corriendo en localhost:3000 con sus categorías
- [ ] **Seguridad**
  .env creado y fuera de cualquier captura

Lo que no llegue hoy, queda de tarea: la Clase 2 arranca directo con el backend.

PARA LLEVARSE

Tres ideas de hoy

1

Primero el mínimo, después los extras.

Una tienda simple que funciona online vale más que una brillante que no anda.

2

La base cuenta la historia del negocio.

Claves foráneas, CHECK y precio_unitario hacen que los datos no mientan.

3

La clave secreta es del servidor.

Va en el .env, nunca en el front ni en GitHub.

Próxima clase: usuarios, catálogo y pedidos en el backend.